The De Bruijn Multiprocessor Network: A Versatile Parallel Processing and Sorting Network for VLSI

MAHESWARA R. SAMATHAM, MEMBER, IEEE, AND DHIRAJ K. PRADHAN, FELLOW, IEEE

Abstract—**It is shown that the binary de Bruijn multiprocessor network (BDM) can solve a wide variety of classes of problems. The BDM admits an N-node linear array, and N-node ring, $(N-1)$-node complete binary trees, $((3N/4)-2)$-node tree machines, and an N-node one-step shuffle-exchange network, where $N(=2^k$, k is an integer) is the total number of nodes.**

The de Bruijn multiprocessor networks are proven to be fault tolerant, as well as shown to be extensible. A tight lower bound of the VLSI layout area of the BDM is derived; a procedure for an area-optimal VLSI layout is also described. It is demonstrated, too, that the BDM is more versatile than the shuffle-exchange (SE) and the cube-connected cycles (CCC).

Recent work has classified sorting architectures into 1) sequential input/sequential output, 2) parallel input/sequential output, 3) parallel input/parallel output, 4) sequential input/parallel output, 5) hybrid input/hybrid output. Such a classification is based not only on the I/O method, but also on the interconnection network, the sorting algorithm, and the type of keys used. Demonstrated here is that the de Bruijn multiprocessor networks (DM) can sort data items in all of the above-mentioned categories. No other network which can sort data items in all the categories is known. Sorting algorithms and time complexities which correspond to each of these categories are given here, as well. Also, it is proven that DM's work as sorting networks, even in the presence of some faults.

Index Terms—**Bisection width, complete binary tree, fault tolerance, fault-tolerant sorting network, Hamiltonian path, sorting network, multiprocessor network, parallel algorithm, parallel sorting network, undirected de Bruijn graph, VLSI layout.**

I. INTRODUCTION

THE SEARCH for computationally efficient multiprocessor architectures that are suitable for VLSI has spawned an increasingly important research area. Several parallel architectures which solve a wide variety of problems have been proposed. These include the linear array [2], [5], the ring [6], the complete binary tree (CBT) [7]–[9], the tree machine (TM) [10], the shuffle-exchange (SE) [11], [14], the cube-connected cycles (CCC) [15], the two-dimensional mesh [16], the even double-exchange [17], the orthogonal trees [18], and the doubly-twisted torus [19].

Manuscript received August 15, 1985; revised December 7, 1986. This work was supported in part by AFOSR 88-0205, as well as in part by NSF MIP 88-05586.

M. R. Samatham is with the Department of Electrical Engineering and Computer Science, Northwestern University, Evanston, IL 60201.

D. K. Pradhan is with the Department of Electrical and Computer Engineering, University of Massachusetts, Amherst, MA 01003.

IEEE Log Number 8823538.

Certain of the real-life problems that must be accommodated have been successfully grouped into various classes [1], [4], [15], [28], [32]. These classifications are important, because statements such as, "all problems in this class have this complexity," inform much more than statements like "this problem is of this complexity" [28]. Included among these classifications are the *pipeline class* [4], the *multiplex class* [32], the *NP-complete class* [24], the *ASCEND* and *DESCEND* classes [15], as well as the *decomposable searching class* [28].

First, problems in the *pipeline class* can be efficiently solved in a pipe (linear array). Depending on the problem, data may flow in one direction or in both directions simultaneously. Matrix–vector multiplication is a typical example of those problems that can be solved by one-way pipeline algorithms [4]. Band matrix–vector multiplication, recurrence evaluation, and priority queues are good representations of problems that can be solved by two-way pipeline algorithms [4].

The *multiplex class* covers a range of problems characterized by [32]: 1) operation on N data operands to produce a single result. 2) Evaluation that can be described by a tree. Evaluation of general arithmetic expressions, polynomial evaluation, etc., is included in this category. The natural computation graph for this paradigm is a tree, whose nodes correspond to operations, and whose edges correspond to data flow between operations. The CBT (complete binary tree) can be used to solve the problems inherent in this class.

Another important class of problems is the NP-complete class [24]. For this class, the CBT can efficiently implement exhaustive search algorithms [1]. Here, time complexity still is exponential.

ASCEND and *DESCEND classes* are comprised of highly parallel algorithms [15]. Here, the paradigm of the algorithms is the iterative rendition of a divide-and-conquer scheme. The input and output are each a vector of $N(=2^k)$ data items; "divide" refers to two subproblems of equal size, where the "marry step" combines the results of two subproblems consisting of the execution of a single operation on the corresponding pairs of data items. That is: assume that input data $D_0, D_1, \cdots, D_{N-1}$ are stored, respectively, in storage location $T[0], T[1], \cdots, T[N-1]$. An algorithm in the DESCEND class performs a sequence of basic operations on pairs of data successively $2^{k-1}, 2^{k-2}, \cdots, 2^1, 2^0$ locations apart. In terms of the above divide-and-conquer model, the marry step involves pairs of 2^0 locations apart. On the other hand, in the

0018-9340/89/0400-0567$01.00 © 1989 IEEE

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

dual class (the ASCEND class), basic operations are performed on the data that are successively $2^0, 2^1, \cdots, 2^{k-1}$ locations apart; the marry step involves pairs of 2^{k-1} locations apart, problems which can be solved in the SE [11] and the CCC [15].

Problems in the *decomposable searching class* can be described as illustrated below [28]. Preprocess a set, F, of N objects into a data structure D such that certain kinds of queries about F can be answered quickly. A searching problem is decomposable if the response to a query Q asking the relation of an object z to the set F can be written as $Q(z, F) = \Delta q(z, f)$, for all f in F, where f is an element in F; Δ is a binary operator which is associative, commutative, and has an identity; and where q is the query asking the relation of the object z to the element f. The TM described in [10] solves this large class of searching problems. This paper demonstrates that multiprocessor networks based on binary de Bruijn graphs referred to as binary de Bruijn multiprocessor (BDM) and general de Bruijn graphs referred to as de Bruijn multiprocessors (DM) can solve all of the above classes of problems efficiently. Specifically, it is shown that these multiprocessor networks can be used as versatile sorting networks.

Sorting is a theoretically interesting problem with a great deal of practical significance [46]. The sorting problem as defined in [42] is described below. We are given N items

$$D_1, D_2, D_3, \cdots, D_{N-1}, D_n$$

to be sorted; we shall call them data items (or records). Each data item D_j has a key K_j which governs the sorting process. The object of such sorting is to determine a permutation $p(1)p(2) \cdots p(N)$ of the data items, which puts the keys in nondecreasing order

$$D_{p(1)} \leq D_{p(2)} \leq \cdots \leq D_{p(N)}.$$

Usually, we output the sorted sequence or the ith smallest item is placed in the ith processor.

A classification method for sorting architectures was presented by Winslow and Chow [47]. The sorters have been classified into the following categories:

1) sequential input/sequential output (SI/SO)
2) parallel input/sequential output (PI/SO)
3) parallel input/parallel output (PI/PO)
4) sequential input/parallel output (SI/PO)
5) hybrid input/hybrid output (HI/HO)

all of which are illustrated in Fig. 1. Note that the classification is based, not only on the I/O method, but also on the interconnection network, the sorting algorithm, and the type of keys used.

This paper demonstrates that the de Bruijn multiprocessor networks can be used to sort elements in all of the five categories. The main advantages are fourfold in having an interconnection network which can sort data items in all of the categories, as the following situations can be handled.

1) Although it is theoretically possible to load the data items in parallel, the number of ports available for I/O may be limited.

2) Even though the I/O ports are available, it may not be possible to load the data items, from the secondary storage, in parallel.

3) Different sets of data may have different types of keys.

4) In practice, there may be faults in the network.

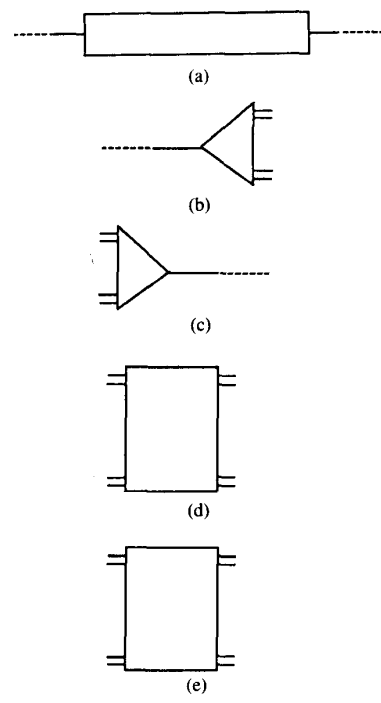


Fig. 1. Categories of sorter architectures (a) Sequential input/sequential output. (b) Sequential input/parallel output. (c) Parallel input/sequential output. (d) Parallel input/parallel output. (e) Hybrid input/hybrid output.

For each of these categories, time complexity and size complexity is given in this paper. The *time complexity* is the worst case time required to sort the data items. The *size complexity* is the number of data items that can be sorted.

The multiprocessor networks based on undirected binary de Bruijn graphs presented here can solve *all* the above-mentioned classes of problems. Also, these networks are shown to be suitable for VLSI implementation. What follows is organized into five sections. Section II describes the binary de Bruijn multiprocessor network (BDM). Section III shows that the BDM admits many basic networks.

The characteristics of both fault tolerance and extensibility for these networks are examined in Section IV. In Section V, a lower bound of the layout area on a VLSI chip, for the BDM, is obtained and a procedure for an area-optimal layout is given. Section VI presents the BDM as a versatile sorting network. Specifically, Section VI describes how the BDM can be used as an SI/SO, PI/SO, and PI/PO sorter. It also describes how the DM can be used as an SI/PO and HI/HO sorter. Finally, Section VII concludes with a demonstration that the BDM is more versatile than the SE and the CCC.

II. The De Bruijn Multiprocessor Networks

A graph is a widely-used model for a multiprocessor network, the nodes of the graph being the processors, the edges being the existing communication links between the proces-

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

sors. The following related definitions and notations are used throughout.

Definitions: The length of a path between two nodes is the number of edges encountered in the path. The *distance* between two nodes is the length of a shortest path between these two nodes. The *diameter* of a network is the largest distance between any two nodes. The degree of a node is the number of edges connected to that node. The *degree* of a network is the largest degree of all the nodes in that network.

Notations: Any vertex (edge) set and its cardinality are represented by $V(E)$ and $|V|$ ($|E|$), respectively. The edge between two vertices, v_1 and v_2, is denoted by $\langle v_1, v_2 \rangle$. Let SE$\{V, E\}$ be the N-node shuffle-exchange graph, where $N = |V| (= 2^k)$. Let SH$(a_{k-1} \cdots a_1 a_0) = (a_{k-2} \cdots a_0 a_{k-1})$ and EX$(a_{k-1} \cdots, a_1, a_0) = (a_{k-1} \cdots, a_1, \bar{a}_0)$ be the shuffle and exchange operations, respectively, where $a_i \in \{0, 1\}, 0 \le i \le k-1$, and $\bar{a}_i$ is the complement of a_i.

Any node whose binary representation is f-bits of y followed by g-bits of $\bar{y}$, followed by h-bits of x, where $y \in \{0, 1\}, \bar{y}$ is the complement of y, and x is a DON'T CARE will be represented as $(y^f \cdot (\bar{y})^g \cdot x^h)$.

A. Undirected de Bruijn Graphs

The undirected de Bruijn graph denoted as [20], [23] DG(d, k) has $N = d^k$ nodes with diameter k and degree $2d$. This corresponds to the state graph of a shift register of length k using d-ary digits. A shift register changes a state by shifting in a digit in the state number in one side, and then shifting out one digit from the other side. If we represent a node by $d_{k-1}d_{k-2} \cdots d_1 d_0$, where $d_j \in \{0, 1, \cdots, (d-1)\}, 0 \le j \le (k-1)$, then its neighbors are represented by $d_{k-2}d_{k-3} \cdots d_0 p$ and $p d_{k-1} d_{k-2} \cdots d_2 d_1$, where $p = 0, 1, \cdots (d-1)$. The multiprocessor network based on the DG(d, k) is called de Bruijn multiprocessor network denoted by DM(d, k), and referred to simply as DM.

B. Undirected Binary de Bruijn Graphs

The DG$(2, k)$ can be obtained as follows. If we represent a node i by a k-bit binary number, say, $i = a_{k-1}a_{k-2} \cdots a_1 a_0$, then its neighbors can be represented as

$$a_{k-2}a_{k-3}a_{k-4} \cdots a_1 a_0 a_{k-1} (i_1) \quad (1)$$

$$a_0 a_{k-1} a_{k-2} \cdots a_2 a_1 (i_2) \quad (2)$$

$$a_{k-2}a_{k-3}a_{k-4} \cdots a_1 a_0 \bar{a}_{k-1} (i_3) \quad (3)$$

$$\bar{a}_0 a_{k-1} a_{k-2} \cdots a_2 a_1 (i_4). \quad (4)$$

The multiprocessor network based on the DG$(2, k)$ is called binary de Bruijn multiprocessor network denoted by BDM(k) and referred to simply as BDM.

Example 2.1: The BDM (3) is shown in Fig. 2.

Neighbors i_1 and i_2 are obtained by a shift-end-around (SEA) operation on i; hence, i_1 and i_2 are called SEA neighbors. Neighbors i_3 and i_4 are obtained by a shift-end-around-complement operation on i; hence, i_3 and i_4 are called SEAC neighbors.

Let SEA(i) represent the shift-end-around operation on i and SEA$^{-1}(i)$ represent the inverse shift-end-around operation on i. Similarly let SEAC(i) represent the shift-end-around-complement operation on i and SEAC$^{-1}(i)$ represent the inverse operation. Note that SEA(SEA$^{-1}(i)$) $= i$ and SEAC(SEAC$^{-1}(i)$) $= i$.

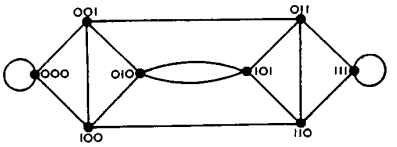


Fig. 2. The DG(2, 3).

The neighbor i_3 is obtained by a shift-end-around-complement (SEAC) operation on i; hence, i_3 and i are called SEAC neighbors. In particular, we denote the above four neighbors of i_1, i_2, i_3, and i_4 as SEA(i), SEA$^{-1}(i)$, SEAC(i), and SEAC$^{-1}(i)$, respectively.

An edge $\langle i, j \rangle$ is called an SEA edge if $j =$ SEA(i) or $j =$ SEA$^{-1}(i)$. Similarly, an edge $\langle i, j \rangle$ is called an SEAC edge if $j =$ SEAC(i) or $j =$ SEAC$^{-1}(i)$.

C. The Proposed Architecture

The BDM is organized as a message-passing network of processors, as opposed to shared memory multiple processors. The proposed BDM, like the cosmic cube [33], therefore employs no switching network between the processors and the memories. Instead, each node has local memory. There are several advantages of the message-passing machine [33], the most important of which is found in the separation of engineering concerns into processor–memory communication and interprocessor communication. The former is reduced because of the close proximity. Also, message-passage machines are simpler as well as more economical than shared-memory machines. Importantly, the greater the number of processors, the greater this advantage becomes. Such an organization is obviously preferred for systems with hundreds of thousands of processing elements.

Also, because the processors are identical, the proposed multiprocessor architecture is regular. Possessing a versatile topology, it is capable of realizing many basic networks, as explained in Section III. Precisely the node architectures required to realize these basic networks is described below.

A block diagram of a processor in the BDM is given in Fig. 3. Each processor has four input/output registers, one for each neighbor. We designate these registers as R(NEIGHBOR), where NEIGHBOR can be SEA, SEA^{-1}, SEAC, or SEAC^{-1}. The processors communicate with their neighbors through these registers; the communication is assumed to be bidirectional. Each processor has an I/O port to communicate with the external world. These ports are used only when the data are available in parallel. When the data are available in serial, it is input and output through the processors 0 and $N-1$.

The complexity of a processing element (PE) depends on the type of applications for which the network is designed. For example, if the network (BDM) is intended to compute only FFT, the PE will only have a multiply-add unit to calculate

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

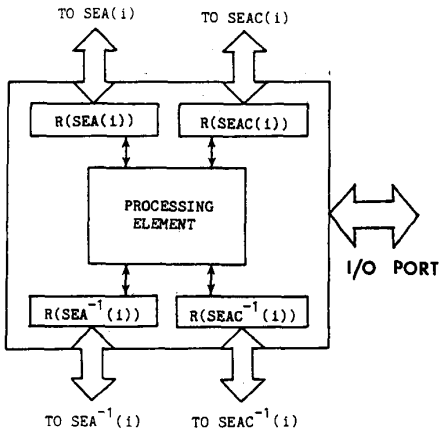


Fig. 3. Block diagram of a processor in the BDM.

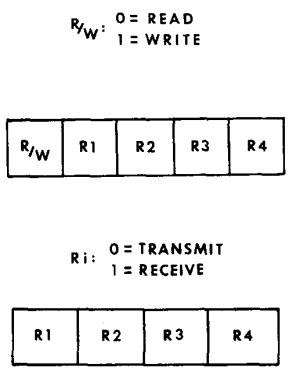


Fig. 4. Format of the control instructions.

weighted sums, and a small local memory to store the weights [38].

All of the processors can work in synchronization with a common clock, called the system clock, or they may work asynchronously. Data movement between the input/output registers and the PE can be synchronized and easily implemented. One possible implementation, illustrated in Fig, 4, requires only 5 bits to specify the operation read/write from/to any combination of registers. For example, 10101 indicates the write operation and that the registers are 2 and 4. Similarly, data movement between the processors can also be achieved easily, by specifying the 4 bits to select the direction of each of the four registers. A bit value, 1, may be used to indicate the receive mode; a value, 0, indicates the transmit mode. For example, 1010 indicates that the registers 1, 3 are in the receive mode and 2, 4 are in the transmit mode. The above-mentioned data movements can be implemented by simple microcode, residing within each processor. The next section depicts the many basic networks embedded in the BDM.

III. Basic Networks in the BDM

In this section, it is demonstrated that the BDM admits many computationally important networks. Specifically, it is shown that it has N-node ring, N-node linear array, $(N-1)$-node CBT's, $((3N/4)-2)$-node TM's, and N-node one-step shuffle-exchange network.

A. A Ring

Definition 3.1: A Hamiltonian cycle is a cycle passing through all the nodes in the graph exactly once.

It is well-known that the BDM has a Hamiltonian cycle. This cycle corresponds to the maximal length shift register sequence [25].

B. A Linear Array

Definition 3.2: A Hamilitonian path is defined as a path passing through all the nodes in the graph exactly one. There are several Hamiltonian paths in the BDM [25].

C. Complete Binary Trees

The directed DG has a tree structure in it [20], [23]. A systematic procedure is given here to construct various CBT's in the BDM.

The BDM has at least four CBT's of height $\log_2 N$. Root nodes of these trees are $y^{k-1} \cdot \bar{y}$ and $\bar{y} \cdot y^{k-1}$, where y is either 0 or 1. The complete binary tree for the root node $y^{k-1} \cdot \bar{y}(\bar{y} \cdot y^{k-1})$ is denoted as CBT $[y^{k-1} \cdot \bar{y}]$(CBT $[\bar{y} \cdot y^{k-1}]$). The following algorithm describes the construction of the CBT $[y^{k-1} \cdot \bar{y}]$.

Algorithm 3.1:

$t := 0$ *Initialization*

Step 1: Let $j := (y)^{k-1-t} \cdot \bar{y} \cdot (x)^t$.

Compute the following: left child of $j := (y)^{k-1-(t+1)} \cdot \bar{y} \cdot (x)^t \cdot 0$, and right child of $j := (y)^{k-1-(t+1)} \cdot \bar{y} \cdot (x)^t \cdot 1$.

Step 2: Repeat step 1 for all possible values of $(x)^t$. There are 2^t possible values of $(x)^t$.

Step 3: If $t = k-1$ then STOP; otherwise increment t and go to Step 1. *end of the algorithm.*

The CBT $[0^3 \cdot \bar{0}]$ and CBT $[1^3 \cdot \bar{1}]$ are shown in Fig. 5(a) and (b), respectively. Similarly, the CBT $[\bar{y} \cdot y^{k-1}]$ can be constructed by replacing step 1 in Algorithm 3.2 by the following:

Step 1a: Let $j := (x)^t \cdot \bar{y} \cdot (y)^{k-1-t}$ Compute the following: left child of $j := 0 \cdot (x)^t \cdot \bar{y} \cdot y^{k-1-(t+1)}$, and right child of $j := 1 \cdot (x)^t \cdot \bar{y} \cdot y^{k-1-(t+1)}$.

end of the Step 1a.

D. Tree Machines

A tree machine (TM) of $(3 \cdot 2^{k-1} - 2)$ nodes is obtained by connecting a $(2^k - 1)$-node CBT and a $(2^{k-1} - 1)$-node CBT back to back. The $(3 \cdot 2^3 - 2)$-node tree machine is shown in Fig. 6.

There are at least four TM's, one for each of the root nodes of input tree $y^{k-1} \cdot \bar{y}$ and $\bar{y} \cdot y^{k-1}$, where y is either 0 or 1 in the BDM(k). The TM for the root node of the input tree $y^{k-1} \cdot \bar{y}(\bar{y} \cdot y^{k-1})$ is denoted by TM$[y^{k-1} \cdot \bar{y}]$(TM$[\bar{y} \cdot y^{k-1}]$). The construction of TM $[y^{k-1} \cdot \bar{y}]$ is described by the following algorithm.

Algorithm 3.2:

Step 1: Construct the CBT $[y^{k-1} \cdot \bar{y}]$ up to the level of $k-1$.

run Algorithm 3.1 by replacing $t = k-1$ *with* $t = k-2$ *in step 3.*

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

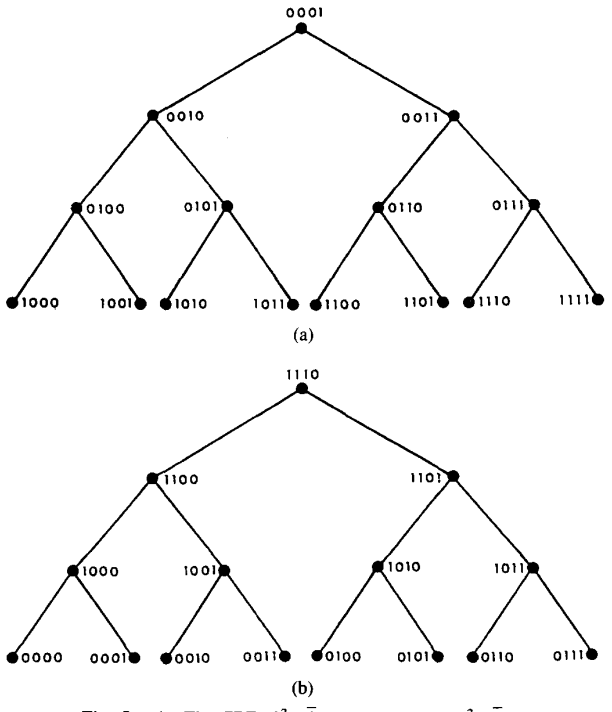


Fig. 5. (a) The CBT $(0^3 \cdot \bar{0})$. (b) The CBT $(1^3 \cdot \bar{1})$.

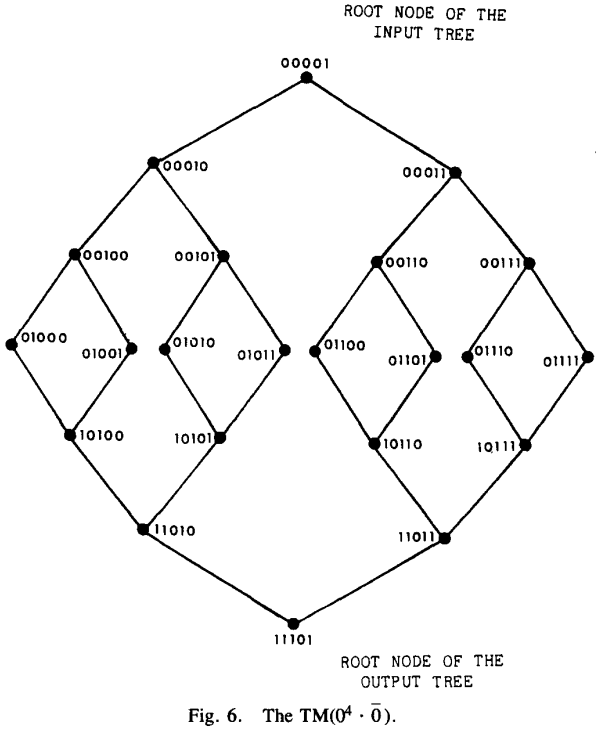


Fig. 6. The TM$(0^4 \cdot \bar{0})$.

Step 2: $t := 0$

Initialization

Step 3: The nodes of the form $(\bar{y}^t \cdot y) \cdot (x)^{k-2-t} \cdot 0$ and $(\bar{y}^t \cdot y) \cdot (x)^{k-2-t} \cdot 1$ are connected to a node of the form $(\bar{y})^{t+1} \cdot y \cdot (x)^{k-2-t}$.

Step 4: Repeat step 3 for all possible values of $(x)^{k-2-t}$. There are 2^{k-2-t} possible values of $(x)^{k-2-t}$.

Step 5: If $t = k-2$ then STOP; otherwise increment t and go to Step 3.

end of the algorithm.

Note that the root node of the output tree is in the form $(\bar{y}^{k-1} \cdot y)$. The TM $[0^4 \cdot \bar{0}]$ is shown in Fig. 6. Similarly we can obtain the TM$[\bar{y} \cdot y^{k-1}]$.

E. One-Step Shuffle-Exchange Network

One can note that the SEA operation is the same as the SH operation, and the SEAC operation is the SH operation followed by the EX operation. Therefore, the BDM can emulate the SE graph. The eight-node shuffle-exchange graph is given in Fig. 7.

Results reported in this section are summarized in Fig. 8. The next section describes that the BDM is fault tolerant and extensible.

IV. Fault Tolerance and Extensibility

A. Fault Tolerance

It has been shown in several instances that these networks have good fault tolerance and self-diagnosability. Some of their characteristics are 1) in the event of a single faulty node, only four additional hops are required to detour around the fault [21], 2) the control information for the bypassing of a faulty node can be incorporated locally with the neighbors of the faulty node [23], 3) the network admits simple distributed fault diagnosis techniques [23].

A different type of fault tolerance question is addressed here: "If there is a fault in a Class 1 graph, can a Class 2 graph or a smaller sized Class 1 graph be extracted?" [31]. The following results for the BDM are obtained from this fault-tolerance point of view.

Lemma 4.1: The CBT$[0^{k-1} \cdot \bar{0}]$ and the CBT$[1^{k-1} \cdot \bar{1}]$ are edge disjoint.

Proof: This is a proof by contradiction. Suppose an edge $\langle i, j \rangle$ exists in both the trees. Let i and j be represented as $(a^i_{k-1}a^i_{k-2} \cdots a^i_0)$ and $(a^j_{k-1}a^j_{k-2} \cdots a^j_0)$, respectively.

From the construction of the CBT's given in Section III-C, leaf nodes of the CBT$[0^{k-1} \cdot \bar{0}]$ are of the form $\bar{0} \cdot x^{k-1}$ and those of the CBT$[1^{k-1} \cdot \bar{1}]$ are of the form $\bar{1} \cdot x^{k-1}$. Furthermore, the nonleafs of CBT$[0^{k-1}\bar{0}]$(CBT$[1^{k-1}\bar{1}]$) are *not* of the form $\bar{0} \cdot x^{k-1}(\bar{1} \cdot x^{k-1})$. First note that i (equally j) must be a leaf of one of the trees, because if i is a nonleaf of CBT$[0^{k-1} \cdot \bar{0}]$ (equally CBT$[1^{k-1} \cdot \bar{1}]$), then to be in CBT$[1^{k-1} \cdot \bar{1}]$ it must be a leaf of CBT$[1^{k-1} \cdot \bar{1}]$. Also, i and j cannot be leaf nodes in the same tree because there is no edge between two leaf nodes. Without loss of generality, assume that i is a leaf node in the CBT$[0^{k-1} \cdot \bar{0}]$ and j is a leaf node in the CBT$[1^{k-1} \cdot \bar{1}]$.

Looking in the CBT$[0^{k-1} \cdot \bar{0}]$, we can conclude that j must be of the form $0 \cdot a^i_{k-1} \cdots a^i_1$, i.e.,

$$a^j_{k-1}a^j_{k-2} \cdots a^j_0 = 0 \cdot a^i_{k-1}a^i_{k-2} \cdots a^i_1. \qquad (5)$$

Similarly looking in the CBT$[1^{k-1} \cdot \bar{1}]$, i must be of the

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

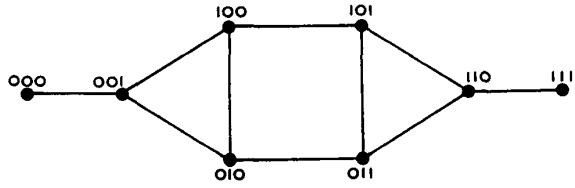


Fig. 7. The eight-node shuffle-exchange graph.

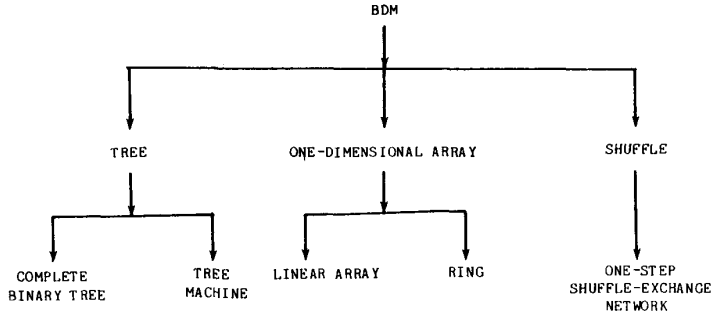


Fig. 8. Various topologies in the BDM.

form $1 \cdot a_{k-1}^{j} \cdots a_1^{j}$, i.e.,

$$a_{k-1}^{i} a_{k-2}^{i} \cdots a_0^{i} = 1 \cdot a_{k-1}^{j} a_{k-2}^{j} \cdots a_1^{j}. \quad (6)$$

From (1) and (2), we can further conclude that $i(j)$ must be of the form $1010 \cdots (0101 \cdots)$, i.e., alternate 1's and 0's starting with 1 (0). However, there are two parallel edges between such nodes. Therefore, the two trees are edge disjoint. Q.E.D.

Corollary 4.1: CBT$[\bar{0} \cdot 0^{k-1}]$ and CBT$[\bar{1} \cdot 1^{k-1}]$ are edge disjoint.

Theorem 4.1: When an edge fails, the BDM will have a fault-free CBT.

Proof: Immediate from Lemma 4.1. Q.E.D.

Lemma 4.2: When a processor fails, the BDM can be configured into a linear array of $N - 1$ nodes.

Proof: Since the BDM has a Hamiltonian cycle, even when a processor fails, it has a Hamiltonian path of the remaining fault-free $(N - 1)$ nodes. Q.E.D.

Corollary 4.2: Even when an edge fails, the BDM can still be configured as a fault-free linear array. Q.E.D.

Lemma 4.3: When a processor fails, the BDM can still be configured as a complete binary tree of height $(k - 1)$.

Proof: When a node of the form $0 \cdot X$, where X is any $(k - 1)$-bit binary number, fails, the BDM has a fault-free CBT$[1^{k-1} \cdot \bar{1}]$ of height $k - 1$. Similarly, if the failed node is of the form $1 \cdot X$, it has a fault-free CBT$[0^{k-1} \cdot \bar{0}]$ of height $k - 1$. Q.E.D.

Lemma 4.4: The DM(d, k) has $\binom{d}{e}$ DM(e, k) graphs in it, where $2 \le d$.

Proof: All the nodes which are represented by only e digits out of d digits form a DM(e, k). There are $\binom{d}{e}$ selections of e digits out of d digits. Q.E.D.

Example 4.1: The DM$(3, k)$ has $\binom{3}{2} = 3$ BDM(k) graphs in it.

Lemma 4.5: Every node i, in the DM(d, k) is in $\binom{d-j}{e-j}$ DM(e, k) graphs, where $j(\le e)$ is the number of different digits in the d-ary digit representation of i.

Proof: A node i is in all those DM(e, k) graphs that are represented by these j digits out of e digits. By fixing these j digits, the remaining $(e - j)$ digits out of the rest of the $(d - j)$ digits can be selected in $\binom{d-a}{e-j}$ ways. Q.E.D.

Lemma 4.6: Let $M(i)$ be the number of different DM(e, k) graphs (of the form in the proof of Lemma 4.4) in which the node i is present. $M(u, u, \cdots, u) > M(i)$, for all $i, i \ne (u, u, \cdots, u)$ and $0 \le u \le (d - 1)$.

Proof: This follows from Lemma 4.5. In the node $(u, u, \cdots, u)$, the value of j is equal to 1. Therefore, this node is in $\binom{d-1}{e-1}$ different DM(e, k) graphs. In all the other nodes, the value of j is greater than 1. Q.E.D.

Let $Z = \binom{d}{e} / \binom{d-1}{e-1} = d/e$ and let $F = \lceil Z - 1 \rceil$.

Theorem 4.2: The DM(d, k) has a fault-free DM(e, k), even if any F nodes (or edges) fail.

Proof: By Lemma 4.5, each node of the form $(u, u, \cdots, u)$ is in $\binom{d-1}{e-1}$ DM(e, k) graphs. This means that a failed node can eliminate at the most $\binom{d-1}{e-1}$ DM(e, k) graphs. But, by Lemma 4.4, there are a total of $\binom{d}{e}$ DM(e, k) graphs. Therefore, F failure will leave at least one fault-free DM(e, k). Q.E.D.

Results obtained in this section are summarized in Table I.

B. Extensibility

Extensibility is an important property which facilitates constructing large-sized systems out of small-sized systems. Here it is shown that the de Bruijn multiprocessor network (DM) is extensible.

There are two methods of extending a DM(d, k) to a DM$(d + 1, k)$. In the first method, extra $[(d + 1)^k - d^k]$ processors are added to the original DM(d, k). In the second method, $\binom{d+1}{d}$ different DM(d, k) are interconnected to ob-

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

TABLE I
FAULT-TOLERANCE RESULTS FOR THE DM

Before the fault	Type of fault	After the fault
BDM(k)	A Communication link	$(2^k - 1)$-node CBT
BDM(k)	A Processor (or a communication link)	$(2^k - 1)$-node linear array
BDM(k)	A Processor (or communication link)	$(2^{k-1} - 1)$-node CBT
DM(d,k)	F Processor	DM(e,k)

tain a DM$(d+1,k)$, provided $d \geq k$. These two methods are described by the following algorithms.

Algorithm 4.1:

Step 1: Represent the processors of the original DM (d,k) with k-digit d-ary numbers, where the d digits are $0, 1, \cdots, (d-1)$.

Step 2: Add $[(d+1)^k - d^k]$ processors, and represent each of these with k-digit $(d+1)$-ary number such that at least one of the k digits is d.

Step 3: Add communication links which satisfy the adjacency relation of the DM$(d+1,k)$.

end of Algorithm 4.1.

Example 4.2: Extension of the DM(2, 2) to the DM(3, 2) using Algorithm 4.1 is shown in Fig. 9. In this figure, the original DM(2, 2) network is indicated by solid lines, processors and communication links that are added are shown by dashed lines.

Algorithm 4.2:

Step 1: Add d more DM(d,k)'s.

Step 2: Represent the processors in all the DM(d,k)'s with k-digit $(d+1)$-ary numbers in such a way that the processors in the same DM(d,k) are represented with the same set of d digits.

Step 3: Treat the processors with the same k-digit representation as a single processor.

Step 4: Add communication links which satisfy the adjacency relation of the DM$(d+1,k)$.

end of algorithm 4.2.

Example 4.3: Extension of the DM(2, 2) to the DM(3, 2) using Algorithm 4.2 is shown in Fig. 10.

The next section presents an area-optimal Thompson model layout for the BDM.

V. Optimal VLSI Layout

In this section, an asymptotically optimal-area layout of the BMD is presented. Here the layout model proposed by Thompson [26] is used. In this model, the chip is divided into a grid of vertical and horizontal tracks, spaced apart by unit intervals. Processors are assumed to occupy unit area and can be located only at the intersections of grid tracks. Wires are routed through the tracks in order to connect pairs of processors. Although a wire in a horizontal track is allowed to cross a wire in a vertical track, pairs of wires are not allowed to overlap any processor to which they are not linked. The layout area is the product of the number of the vertical and the number of the horizontal tracks in the layout.

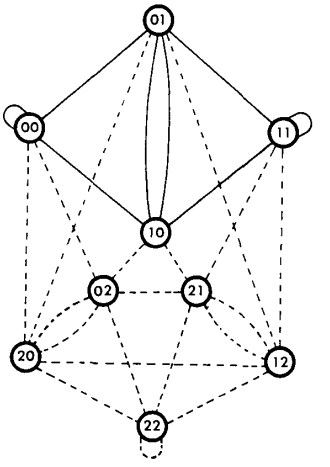


Fig. 9. Extension of the DM(2, 2) to the DM(3, 2) using Algorithm 4.1.

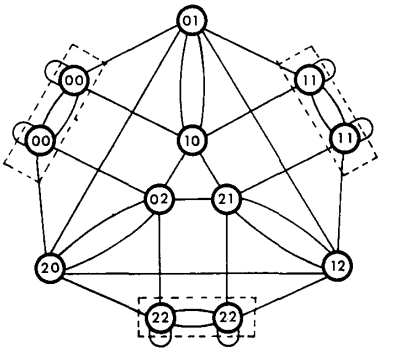


Fig. 10. Extension of the DM(2, 2) to the DM(3, 2) using Algorithm 4.2.

A. Lower Bound on Area

First, a lower bound of the layout area is derived.

Lemma 5.1: If the distance between two nodes, v_1 and v_2, in the SE is greater than 2, then $\langle v_1, v_2 \rangle \notin E$, where E is the edge set of the BDM.

Definition 5.1: A bisection of a graph $G = \{V, E\}$ is the subset B of E and a partition of V into two sets, V_1 and V_2, such that $|V_1| = |V_2|$; B contains all the edges of G connecting the vertices in V_1 with the vertices in V_2. The width of a bisection is $|B|$ and the minimum bisection width of a graph is the least width of any bisection of the graph denoted by $|B|_{\min}$.

Lemma 5.2: $|B|_{\min}$ of the BDM is $\Theta(N/\log N)$.

Proof: Any N-node graph which can compute an N-point FFT in $O(\log N)$ steps has a $|B|_{\min}$ of $\Omega(N/\log N)$ [26]. The BDM can compute an N-point FFT in $O(\log N)$ steps [38]. Therefore, $|B|_{\min}$ of the BDM is $\Omega(N/\log N)$. Next it is shown that $|B|_{min}$ of the BDM is $O(N/\log N)$.

Consider the minimum bisection of the SE. Let $v_{11}, v_{12}, \cdots, v_{1(N/2)}$ be the vertices in V_1, and $v_{21}, v_{22}, \cdots, v_{2(N/2)}$ be the vertices in V_2. It may be noted that since the bisection width of the SE is $\Theta(N/\log N)$ both N and m are integers equal to $c \cdot (N/\log N)$, c is a constant

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

greater than 0. The vertices in V_1 can be placed into three levels as follows:

$$V_{1(\text{LEVEL0})} = \{v_{11}, v_{12}, \cdots v_{1m}\} \quad (7)$$

$$V_{1(\text{LEVEL1})} = \{v_{1i} : \langle v_{1j}, v_{1i}\rangle \in E, i > m \text{ and } j \le m\} \quad (8)$$

$$V_{1(\text{LEVEL2})} = \{v_{1i} : \langle v_{1j}, v_{1i}\rangle \notin E, i > m \text{ and } j \le m\}. \quad (9)$$

Similarly, the vertices in V_2 can be partitioned into $V_{2(\text{LEVEL0})}, V_{2(\text{LEVEL1})}, V_{2(\text{LEVEL2})}$. The following can be concluded: 1) $|V_{1(\text{LEVEL0})}|, |V_{2(\text{LEVEL0})}| = c \cdot (N \log N)$, since the $|B|_{\min}$ of the SE is $O(N \log N)$. 2) $|V_{1(\text{LEVEL1})}| \le 2 \cdot c \cdot (N \log N)$ and $|V_{2(\text{LEVEL1})}| \le 2 \cdot c \cdot (N \log N)$, since the degree of vertices in SE is less than or equal to 3.

Now, the BDM is considered with the above partition of vertices. By Lemma 5.1, the edges in the BDM which connect a node in V_1 and a node in V_2 exist only between the vertices $V_{1(\text{LEVEL0})} \cup V_{1(\text{LEVEL1})}$ and the vertices in $V_{2(\text{LEVEL0})} \cup V_{2(\text{LEVEL1})}$. Since the degree of each vertex in BDM is four, $|B|_{\min}$ of the BDM is $O(N/\log N)$. Therefore, $|B|_{\min}$ of the BDM is $\Theta(N/\log N)$. Q.E.D.

Theorem 5.1: Any VLSI layout of the BDM requires at least $\Omega(N^2/\log^2 N)$ area.

Proof: Thompson [26] has proved that the layout area of any graph is $\Omega(|B|^2_{\min})$. The theorem follows from the Lemma 5.2. Q.E.D.

B. Optimal-Area VLSI Layout

It is shown here that any layout procedure for the SE network may be used to layout the BDM network in comparable area. An asymptotically optimal layout for the SE network is given in [14]. This yields an asymptotically optimal layout for the BDM network. The following algorithm lays out BDM(k).

Algorithm 5.1

Step 1: Layout the 2^{k+1}-node SE network in $O([2^{k+1}/(k+1)]^2)$ area.

Step 2: Double the lengths and widths of all nodes and edges, so each node occupies a 2×2 area and double the length of all edges.

Step 3: Duplicate each EX edge.

Step 4: For each node labeled $X^k \cdot 1$, replace the node with connections between the two EX edges and the two SEA edges.

Step 5: Strip off the last (bit a_0) from the labels of all nodes.

end of the algorithm.

Example 5.1: Fig. 11(a) shows a fragment of a 32-node SE network. Fig. 11(b) shows the layout after applying step 3 of Algorithm 5.1. Fig. 11(c) shows the final product of Algorithm 5.1.

Example 5.2: Fig. 12 shows a layout for the 16-node SE network. Applying Algorithm 5.1 to this layout yields the layout in Fig. 13 for the BMD (3) network.

Theorem 5.2: For the BDM (k), when k is large, Algorithm 5.1 gives an area-optimal layout.

Proof: Kleitman [14] gives a layout for the SE network

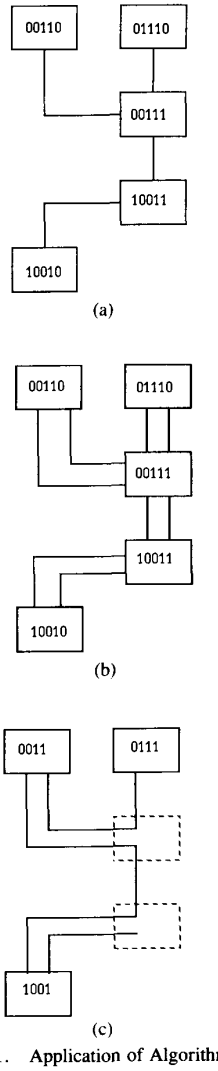


Fig. 11. Application of Algorithm 5.1.

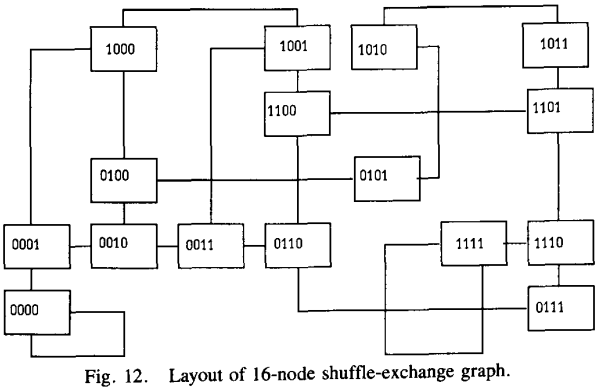


Fig. 12. Layout of 16-node shuffle-exchange graph.

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

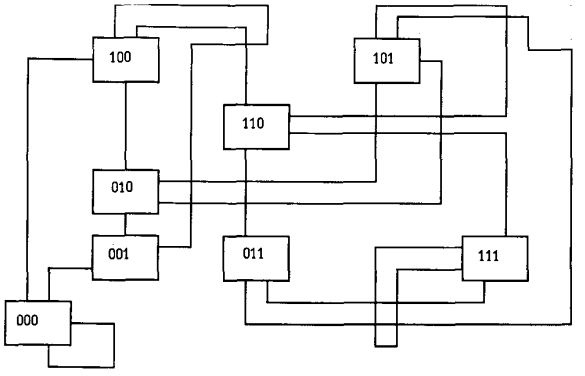


Fig. 13. Layout of eight-node de Bruijn graph.

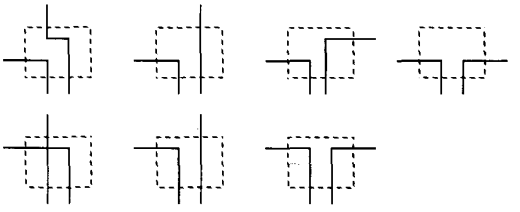


Fig. 14. Substitution of edges for a node in step 4.

bounded by $c \cdot N^2/\log^2 N$, for some constant c. Step 1 requires the layout for a $2N$-node SE network. This occupies area $\leq c \cdot (2N)^2/\log^2(2N) \leq 4c \cdot N^2/\log^2 N$. Step 2 increases the area by a factor of two in both dimensions, so the area used is $\leq 16c \cdot N^2/\log^2 N$, which is $O(N^2/\log^2 N)$. The area does not increase with any further step. The connections made in step 4 can be made in the 2×2 area of a node. All cases (ignoring symmetric cases) are shown in Fig. 14.

The layout method of Algorithm 5.1 is very crude, but much of the area wasted could be recovered by fine tuning using a plowing variety of program. Furthermore, the algorithm's output could be "shaken" and annealed to obtain a compact layout. Q.E.D.

VI. The De Bruijn Multiprocessor Network as a Sorting Network

A. The BDM as SI/SO Sorter

The ladder sorter, introduced by Chen *et al.* [41], is a typical example of an SI/SO sorter, consisting of a chain of shift registers with comparators between links in the chain. Data items are loaded sequentially; as the items are loaded, they are interchanged among the shift registers so that as soon as the last item is loaded, the sorted data begin emerging sequentially from the ladder. The size of the ladder and the sorting time, which includes the time for loading, are both $O(N)$.

The SI/SO sorter can be further classified as 1) two port, and 2) single port. In a two-port SI/SO sorter, data items are input through one port, and output through some other port. In a single-port SI/SO sorter, data items are input as well as output through only one port. Interestingly, the BDM can realize both a two-port SI/SO and a single-port SI/SO sorter.

B. The BDM as a Two-Port SI/SO Sorter

As shown earlier, the BDM has a Hamiltonian path between the processors 0 and $(N-1)$.

Definition: The right (left) neighbor of a node i is the node which is on the right (left) side of the node i, in the Hamiltonian path from the node 0 to $N-1$ (node 0 on the left and node $N-1$ on the right) and is denoted by $R(i)$ $(L(i))$.

If each processor has a comparator which can compare two data items, and executes the following algorithm, it works as a two-port SI/SO sorter. The main idea behind the following algorithm is that whenever a processor i has two data items, it sends the smaller of the two data items to $R(i)$. If this processor has only one data item and it has not received another item from $L(i)$ then it sends this item to $R(i)$.

Algorithm for Processor i

```
Start:Count1: = 0
      Count2: = 0
Initializations

Count1 keeps track of how many data items have passed
through this processor and Count2 keeps track of how
many data items are now with this processor

Loop:IF a data item is received from L(i)
            then begin
                 Count1: = Count1+1
                 Count2: = Count2+1
                 end
            end if
If Count2=2 then begin
            send smaller of the data items to R(i)
            Count2: = Count2-1
            end
end if
If (Count1 = N and Count2 = 1) then begin
            send the data item to R(i)
                 goto Start
                 end
            else goto loop
end if
end of algorithm.
```

Lemma 6.1: The worst case time required to sort N data items is $3N-1$, which includes loading and unloading time.

Proof: The loading phase requires N time units; the worst case time to receive the smallest item at the output port is N. The remaining $N-1$ data items can be received in $N-1$ time units. Q.E.D.

Example 6.1: The sorting of four data items by the BDM as a two-port SI/SO sorter is shown in Table II.

Note that the above sorter is different from the sorter presented in [48]; there, a global bus is used to pass data to each cell, whereas ours uses none.

C. The BDM as a Single-Port SI/SO Sorter

The BDM was shown earlier to contain a Hamiltonian cycle. If each processor has a comparator, and executes the following algorithm, it works as a single-port SI/SO sorter.

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SORTING OF FOUR DATA ITEMS USING TWO-PORT SI/SO SORTER

Time Unit		Contents of the Processors				
	Input	Proc. # 0	Proc. # 1	Proc. # 2	Proc. # 3	Output
t_1	1,2,3	4				
t_2	1,2	3,4				
t_3	1	2,4	3			
t_4		1,4	2,3			
t_5		4	1,3	2		
t_6			4,3	1,2		
t_7			4	3,2	1	
t_8				4,3	2,1	
t_9				4	3,2	1
t_{10}					4,3	2,1
t_{11}					4	3,2,1
t_{12}						4,3,2,1

Algorithm 6.1:

Step 1: Each processor executes the algorithm given in Section VI-B to load a maximum of $2N$ data items. (After the loading phase, each processor has two data items.)

Step 2: Each processor sends the larger of the two elements to $L(i)$.

Step 3: Repeat step 2 until all the processors are empty.

end of algorithm.

Lemma 6.2: The worst case time to sort N data items is $3N$ time units, and that to sort $2N$ data items is $4N$ time units.

Proof: Sorting N data items is similar to that of the two-port SI/SO, except that the last data item has to go through one additional processor.

In the case of $2N$ data items, loading the data requires $2N$ time units. Odd–even transposition sorting requires N time units; unloading the data requires $2N$ time units.

D. The BDM as PI/SO Sorter

The tree sorter is a typical example of the PI/SO sorter, developed by Song [8]. Parallel data are fed to the leaves of a binary tree; pairs of leaves are compared simultaneously and the larger (or smaller) of the two sons is propagated to the father node. Continuing in this fashion, after $\log_{2N}$ time units, the sorted sequence starts emerging from the root of the tree. The size and time complexity are $O(N)$ and $O(N + \log_2 N)$, respectively.

The BDM has four complete binary trees, CBT[$y^{k-1} \cdot \bar{y}$] and CBT[$\bar{y} \cdot y^{k-1}$], where y is either 0 of 1. If each processor of the BDM has a comparator which can compare two data items and executes the following algorithm, it works as a PI/SO sorter. Data items are input through the processors, i, $N/2 \le i \le (N-1)$. Initially, each of these processors contains two data items.

Algorithm for Processor i: $i = a_{k-1}a_{k-2} \cdots a_1a_0$

begin

If a processor receives a request from its parent, **then** send the requested data items to its parent.

If a processor has two data items, send the smaller key value to its parent.

If a processor has one or no data items, compare the keys of its sons and request for the data item which has a smaller key value.

end.

end of algorithm

Lemma 6.3: The size and time complexity of the above sorter are N and $N + \log_2 N$, respectively.

E. The BDM as PI/PO Sorter

Stone [11] has given an $O(\log_2 N)$ time algorithm to execute Batcher's bitonic sorting method [11] on a perfect shuffle interconnection network. The following is an adaptation of the algorithm given in [40]. The key difference is that the algorithm given here is distributed in the sense that each processor, i, $0 \le i \le (N-1)$, executes the algorithm without the use of a centralized controller.

Algorithm for Processor i:

$i = a_{k-1}a_{k-2} \cdots a_0$

```
begin
Procedure Shuffle1 (R)
    begin
        send R to SEA(i)
    end;
end of procedure Shuffle 1
Procedure Shuffle2 (Data)
begin
    send Data to SEA(i) and SEAC(i)
end;
end of procedure Shuffle 2
Procedure Compare-Exchange
    begin
    Min := minimum of {R(SEA^-1(i)), R(SEAC^-1(i))}
    Max := maximum of {R(SEA^-1(i)), R(SEAC^-1(i))}
    if Mask := 0 then if a0 = 0 then Data := Min
                 else Data := Max
                 else if a0 = 0 then Data := Max
                 else Data := Min
    end if;
end;
end of procedure Compare-Exchange
Main program begins
    R := a0
a0 is the least significant bit of i
    Mask := R
Mask bit determines the behavior of Compare-Exchange operation
    For j := 1 to k do
    begin
        Mask := Mask ⊕R
        Shuffle1 (Mask)
    end
    Shuffle2 (Data)
    Compare-Exchange
    For j := 1 to k - 1 do
    begin
    Shuffle (R)
    Mask := Mask ⊕R
    For l := 1 to k - 1 - j do
    Shuffle1 (Mask)
    Shuffle2 (Data)
```

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

```
    For l := k − j to k − 1 − j do
    begin
        Shuffle1 (Mask)
        Shuffle2 (Data)
        Compare–Exchange
    end
end.
```

end of the algorithm.

Lemma 6.4: The size and time complexities of the above sorter are N and $O((\log_2 n)^2)$, respectively.

F. The BDM as $O(\log^2 N)$-Time PI/PO Sorter

Examples of the PI/PO sorters are the network sorters proposed by Batcher [40]. These networks include the merge sorter and the bitonic sorter. Both sorters require $N \log_2^2 N$ comparators and $O(\log_2^2 N)$ time units. Muller and Preparata [44] have shown the sorting times of $O(\log_2 N)$ are possible for PI/PO sorters, if $O(N^2)$ comparators are used. Recently, Leighton [43] provided a construction of a bounded-degree $O(N)$-node network that sorts N numbers in $O(\log N)$ steps. An example of the SI/PO is the distributed tree sorter [47]. It is shown that by replacing the comparators by distributors it is possible to improve the sorting time [47]. A tree of distributors can be constructed by placing one distributor at the root of the tree, d distributors at level one of the tree, and so forth, with d^p distributors at level p of the tree. A data item is inserted at the root and passed down the levels until it reaches the output level. The result is an SI/PO sorter with size and time complexities of $O((N-1)/(d-1))$ and $O(N + \log_d N)$, respectively.

Nassimi and Sahni [45] have given an $O(p \cdot \log N)$ algorithm to sort N data items, on perfect shuffle computers, with $N^{(1+1/p)}$ processing elements, $1 \leq p \leq \log N$. This algorithm can be easily adapted to execute on the BDM. This means that the BDM of N^2 processors can sort N data items in $O(\log N)$ time.

Even though the works of Ajtai, Komlos, and Szemeridi [39] and Leighton [43] are milestones in the history of sorting, the constants involved in their results are quite high. Therefore, because of this, we conclude that for the reasonable values of N the BDM of N processors may sort data items faster than those that are given in [39] and [43].

G. The DM as a SI/PO Sorter

The DM of $N(= d^k)$ processors has d-ary trees of height $\log_d N$ [23]. We are interested in the tree whose root node is $(0, 0, 0, \cdots 0)$. The DM as a tree for the root node $(0, 0, \cdots, 0)$ is shown in Table II.

Definition: The jth neighbor of a node i is the node which is obtained by shifting digit j into the most significant digit and by omitting the least significant digit of i.

In this tree, the children of the node i are all jth neighbors, $j = 0, 1, \cdots, (d-1)$. There are $(d-1)$ processors in level 2, $d(d-1)$ processors in level 3 and so on, $d^{k-1}(d-1)$ processors in level k. If each processor in the DM has a distributor and executes the algorithm given below, it works as an SI/PO sorter.

The level of node i denoted by LEVEL(i), in the above-mentioned trees, can be expressed as LEVEL(i) = k + 2−POSITION(i), where $k = \log_d N$ and POSITION(i) is the position where the first nonzero digit exists when counted from right to left in i, represented by a k-digit d-ary number.

Example: POSITION($000 \cdots 00$) = $k + 1$ and POSITION($XX \cdots X1$) = 1, where X is a DON'T CARE.

We make the following assumptions.

1) There are N data items.
2) All the keys are represented by k-digit d-ary numbers.
3) No two keys have the same value.

Algorithm for Processor i

```
begin
    Determine the level, LEVEL(i)
    Initialization
    Whenever a data item is received do
    begin
    If ((LEVEL(i) = k + 1) OR (key value = i))
    then retain this item
    else
        begin
            Determine the digit value, D(key), of the
            key in the position LEVEL(i) starting from
            the right, when the key is represented by a
            k-digit d-ary number
        Send data item to D(key)th neighbor.
        end if
end.
```

end of algorithm

The output is received from all the nodes in parallel. The data item with key (X) will appear at the node (X).

Lemma 6.5: The size and time complexities are N and $N + \log_d N$, respectively.

H. The DM as an HI/HO Sorter

The parallel balanced tree sorter (PBT), described in [47], is an example of an HI/HO sorter. In a balanced tree (BT), all subtrees have the same height. Each node in a balanced tree contains $(d-1)$ values and the "nearest" value is used to determine which son node is used for the next comparison node. Sooner or later, it reaches a node where a match is found, or it reaches the bottom of the tree. If we have SBT sorters working in parallel, each sorting a separate subtree, the sort time should be divided by S.

Implementations of BT and PBT sorters are described in [47]. Now, a brief description of the workings of the DM as a balanced tree (BT) sorter is given below.

Conceptually, a BT sorter consists of a base d comparator (the design given in [23]), and a set of buckets. The base d comparator inputs an item, compares the item to all $(d-1)$ node values simultaneously, determines the "nearest" node value, and assigns the item to a bucket which is associated with the node value.

In the DM, if each processor has a base d comparator and executes the algorithm described below, it works as a BT sorter. The following assumptions are made.

1) The total of data items is N

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

2) The exact node values ($d-1$ in number) of each processor are known and these values are selected so that the tree is always balanced. Let these values for processor i be $D_j(i), 1 \le j \le (d-1)$.

3) Data items are input through the node $(0, 0, \cdots 0)$, adjacent to the root and each data item is output through the corresponding node.

Algorithm for Processor i

begin
Procedure d-Compare (data)
begin
CASE key (data) **of:**
begin
$D_j(i) \le$ key (data) $\le D_{j+1}(i), 1 \le j \le (d-1)$; send data to jth neighbor.
key (data) $\le D_{d-1}(i)$: send data to 0th neighbor.
key (data) $> D_{d-1}(i)$: send data to $(d-1)$st neighbor.
end CASE
end
end of procedure d-Compare (data)
Whenever a data item is received **do**
If POSITION $(i) \neq 1$ **then** d-compare (data)
else stop
end if
end
end of algorithm.

The size and time complexities of the above sorter are N and $(N + \log_d N)$, respectively. In the above sorter, it is assumed that leaf nodes can sort incoming data.

I. The DM as PBT Sorter

Theoretically, if we have d BT sorters working in parallel, each sorting a separate subtree, the sort time should be divided by d. The problem is to store the subtrees in separate memories and to avoid conflicts during the first pass. In the DM, this can be achieved by inputing the data through processors at level 2($(d-1)$ processors) and by arranging the memory in the root node $(0, 0, \cdots 0)$ into $2(d-1)$ separate memory banks. The reason for using $2(d-1)$ memory banks is explained in [47]. The memories are assumed to have multiple ports, one port for each processor at level 2. The size of the memory unit is selected so that there will not be any overflow. Two consecutive memory banks constitute a memory unit. The jth data item in the lth memory unit is denoted by $M(l, j), 1 \le l \le (d-1), l \le j \le (N/(d-1))$.

The DM works as a PBT in two passes. In the first pass, as the data items are input through the processors in level 2, they are sent to one of the memories in the root node, after comparing to each of its node values ($2(d-1)$ in number). These node values are selected so that in the second pass, the tree remains balanced.

In the second pass, the root node sends the corresponding data items to all of the nodes in level 2, in parallel. The algorithm executed by the root node, in the second pass, is given below. All of the processors, except the root node and the leaf nodes, execute the algorithm given in the case of the BT sorter. The base d comparator hardware in the leaf nodes is used to sort the incoming data items as they are loaded into the comparator. A hybrid output of d data items from each processor is yielded. Note that $(d-1)$ data items are input in parallel.

Algorithm for the Root Node:

begin
for $j := 1$ to N **do**
for $l := 1$ to d **in parallel do**
Send $M(l, j)$ to $(l-1)$st neighbor
end.

Lemma 6.6: The size and time complexities of the above sorter are N and $2(N/d) + \log_d N + c$, respectively, where c is a variable which depends on the queueing time of the multiport memory.

The results obtained in this section are summarized in Table III.

J. The DM as a Sorting Network Under Faults

This section shows that the de Bruijn multiprocessor network works as a sorting network even in the presence of some faults. First, we study the BDM.

K. The BDM as a Sorting Network Under Faults

Here, assuming that the BDM is capable of sorting in PI/PO mode when there are no faults, we obtain the following results.

Theorem 6.3: When a communication link fails, the BDM can still be used as a PI/SO sorter.

Proof: Proof follows from the Lemma 6.1. Q.E.D.

Theorem 6.4: When a processor fails, the BDM can be used as a PI/SO sorter of size complexity 2^{k-1}.

Proof: Even if a processor fails, a complete binary tree of height k has a complete binary tree of height $k-1$.Q.E.D.

Theorem 6.5: When a node fails, the BDM can be used as a two-port SI/SO sorter.

Proof: Since the BDM has a Hamiltonian cycle, when a processor fails, it will have a Hamiltonian path of length $(N-1)$. Q.E.D.

L. The DM as a Sorting Network Under Faults

Here, we present some results for the general de Bruijn multiprocessor networks.

Theorem 6.6: The SI/SO sorter of size complexity d^k can be used as an SI/PO sorter of size e^k, even when there are F faults, where F is as defined in Section IV-A.

Proof: Proof follows from the Theorem 2. Q.E.D.

Theorem 6.7: When $p, 1 \le p \le (d-1)$, processors fail, the faulty DM can be used as a BT sorter.

Proof: The PBT sorter is essentially d BT sorters working in parallel. When $(d-1)$ processors fail, still we have a BT sorter. Q.E.D.

These results are summarized in Table IV.

VII. Comparison and Conclusions

It is shown here that the BDM admits 1) an N-node ring, 2) an N-node linear array, 3) $(N-1)$-node CBT's, 4)

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

TABLE III
SIZE AND TIME COMPLEXITIES OF THE DM AS A SORTING NETWORK

Category	Size complexity	Time complexity
A. SI/SO		
(a) Two port	N	$3N$
(b) Single port	N	$3N$
	$2N$	$4N$
B. PI/SO		
	N	$N + \log_2 N$
C. PI/PO		
(a) N Processor	N	$0((\log_2 N)^2)$
(b) N^2 Processor	N	$0(\log_2 N)$
D. SI/PO	N	$N + \log_d N$
E. HI/HO	N	$2(N/d) + \log_d N + c$

TABLE IV
THE DM AS A SORTING NETWORK UNDER FAULTS

Before the fault			After the fault	
Sorter	Size Complexity	Type of Fault	Sorter Complexity	Size
PI/PO	2^k	A communication link	PI/SO	2^k
PI/PO	2^k	a processor	PI/SO	2^{k-1}
PI/PO	2^k	a processor	SI/SO	$2^k - 1$
SI/PO	d^k	F processors	SI/PO	n^k
HI/HO	d^k	(d-1) processors	SI/PO	e^{k-1}

$((3N/4) - 2)$-node TM's, and 5) an N-node one-step shuffle-exchange network. Thus, the BDM is quite useful for constructing a processor network that performs computations for 1) the pipeline class, 2a) the multiplex class, 2b) the NP-complete class, 3) the decomposable searching class, and 4) the ASCEND and DESCEND classes of problems. The BDM can also be used as an efficient VLSI dictionary machine [37], since it has a complete binary tree. A tight lower bound of the VLSI layout area of the BDM is given, and a procedure for an area-optimal VLSI layout is derived, as well. It is also shown that the de Bruijn multiprocessor network is fault tolerant in its ability to gracefully degrade in the event of certain faults, and these networks are shown to be extensible.

In comparing the BDM to the SE and the CCC, the following may be observed. Although the BDM is similar to the SE, the former is twice as fast as the latter in executing the ASCEND and DESCEND classes, the multiplex class, the NP-complete class, and the decomposable searching class of problems. This is chiefly because the BDM is obtained by a one-step shuffle-exchange (SEAC) operation. Since in VLSI the communication is costly, this will substantially improve performance. Even more importantly, the SE does not have a Hamiltonian cycle, and it is not even known whether it can have a Hamiltonian path. Consequently, the pipeline class of algorithms cannot be executed on the SE. It is also not known whether the CCC can be used to efficiently solve pipeline, NP-complete, or decomposable searching classes of problems, because it is not known whether the CCC has a linear array, a complete binary tree, or a tree machine. Interestingly, in the case of the ASCEND and DESCEND classes, the BDM can be as much as 2 times faster than the CCC. This is due to the fact that in the case of simple algorithms [15] for example, the CCC requires $O(k - r) + O(2^r)$ steps, where r is the smallest integer for which $k \leq (r + 2^r)$ and 2^k is the total number of processors in the network. But 2^r is approximately equal to k.

It can be noted that one possible shortcoming of the BDM, as compared to the CCC, is that the Thompson model layout of the BDM is not as regular as that of the CCC. In practice, though, one has to introduce some fault tolerance (like spare processors, buses, switches) into the network [27], [36]. This factor may minimize the significance of the regularity of the layout of the original interconnection network.

In comparing the BDM to the binary cube network [33], it may be noted that it is known [49] that the cube network can emulate linear arrays, meshes, and binary trees (if we allow one of the nodes used as a connecting node). However, it appears that the cube network cannot embed or emulate the shuffle-exchange network easily. One of the useful networks that the cube can emulate is the mesh of trees [50]. At this time it is not known whether the BDM can also emulate the mesh of trees. The advantage the BDM has over the cube is that the number of connections per node does not grow with the size of the network.

In summary, the conclusion is that the BDM can be effectively used as a general purpose parallel processing network for VLSI. It can also be used as a fault-tolerant add-on processor, for execution of multiple applications, since the network can be dynamically restructured to fit the applications.

Furthermore, this paper has shown that the architectures based on the DG's can be used to sort data items in 1) sequential input/sequential output, 2) parallel input/sequential output, 3) parallel input/parallel output, 4) sequential input/parallel output, and 5) hybrid input/hybrid output modes. Sorting algorithms that correspond to each of these categories are given. The algorithms are distributed in the sense that these are executed by individual processors without any centralized controller.

Time complexities and size complexities that correspond to each of the categories are tabulated in Table III. It can be concluded that the architectures based on the DG's achieve the previously known best upper bound times. Also these networks are shown to be fault-tolerant sorting networks. Finally, a comparison of binary cube and de Bruijn graph is illustrated in Table V.

Finally we pose an open question: "Is it possible to sort N data items in $O(\log N)$ time in the BDM of N (or $N \log N$) processors?"

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

TABLE V
DE BRUIJN GRAPHS—AN ATTRACTIVE ALTERNATIVE TO THE BINARY CUBE

	Binary-Cube	Binary-DeBruijn Graph
Number of Nodes	2^n	2^n
Number of Connections Per Node	n	4
Node Size	Grows with the Size of the Network	Constant
Longest Path Length	n	n
Routing	Easy Self-Routing	Easy Self-Routing
Fault-Tolerance	Yes	Yes
Routing with Node/Link Fault	Easy	Easy
Longest Path Length with Single Fault	At Most (n+1)	At Most (n+4)
Allows Easy Detours Around Faults	No	Yes
Extensibility	Difficult	Difficult
Emulation of: Binary Tree	Difficult	Easy (even w/faults)
Tree Machine (Back to Back Binary Tree)	Not Known	Easy
Shuffle-Exchange	Not Possible	Easy
Linear Array	Easy	Easy (even w/faults)
Mesh	Yes (even w/faults)	Not Known

ACKNOWLEDGMENT

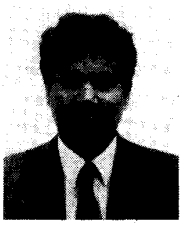
The authors wish to thank F. T. Leighton of M.I.T., V. K. P. Kumar of U.S.C., and F. Meyer of the University of Massachusetts for helpful suggestions and useful hints in obtaining proofs.

REFERENCES

[1] C. Mead and L. Conway, *Introduction to VLSI Systems.* Reading, MA: Addison-Wesley, Oct. 1980.
[2] H. T. Kung and C. E. Leiserson, "Systolic arrays for VLSI," in *Introduction to VLSI Systems,* C. Mead and L. Conway, Eds. Reading, MA: Addison-Wesley.
[3] R. P. Brent and H. T. Kung, "Systolic VLSI arrays for polynomial GCD computation," Tech. Rep., Carnegie-Mellon Univ., Mar. 1982.
[4] H. T. Kung, "The structures of parallel algorithms" in *Advances in Computers, Vol. 19.* New York: Academic, 1980, pp. 65–112.
[5] P. E. Danielsson, "Serial/parallel convolvers," *IEEE Trans. Comput.,* vol. C-33, pp. 652–667, July 1984.
[6] H. T. Kung and M. S. Lam, "Fault-tolerant and two-level pipelining in VLSI systolic arrays," in *Proc. MIT Conf. Advance Res. VLSI,* Jan. 1984.
[7] S. A. Browning, "The tree machine: A highly concurrent computing environment," Ph.D. dissertation, Dep. Comput. Sci., California Instit. Technol., Jan. 1980.
[8] S. W. Song, "A highly concurrent tree machine for data base applications," in *Proc. 1980 Int. Conf. Parallel Processing,* Aug. 1980, pp. 259–268.
[9] E. Horowitz and A. Zorat, "The binary tree as an interconnection network: Applications to multiprocessor systems and VLSI," *IEEE Trans. Comput.,* vol. C-30, pp. 247–253, Apr. 1981.
[10] J. Bently and H. T. Kung, "A tree machine for searching problems," in *Proc. 1979 Int. Conf. Parallel Processing,* Aug. 1979, pp. 257–266.
[11] H. S. Stone, "Parallel processing with the perfect shuffle," *IEEE Trans. Comput.,* vol. C-20, pp. 153–161, Feb. 1971.
[12] T. Lang, "Interconnection between processing and memory modules using the shuffle-exchange network," *IEEE Trans. Comput.,* vol. C-25, pp. 55–66, Jan. 1976.
[13] D. S. Parker, "Notes on shuffle-exchange type switching networks," *IEEE Trans. Comput.,* vol. C-29, pp. 213–222, Mar. 1980.
[14] D. Kleitman, F. T. Leighton, M. Lepley, and G. L. Miller, "New layouts for the shuffle-exchange graph," in *Proc. 13th Annu. ACM Symp. Theory Comput.,* May 1981, pp. 278–292.
[15] F. P. Preparata and J. E. Vuillemin, "The cube-connected-cycles: A versatile network for parallel computation," *Commun. ACM,* pp. 300–309, May 1981.
[16] C. D. Thompson and H. T. Kung, "Sorting on a mesh-connected parallel computer," *Commun. ACM,* vol. 20, pp. 263–271, 1977.
[17] W. E. Leland, "Density and reliability of interconnection topologies for multicomputers," Ph.D. dissertation, Univ. Wisconsin, Madison, May 1982.
[18] D. D. Nath, S. N. Maheshwari, and P. C. P. Bhatt, "Efficient VLSI networks for parallel processing based on orthogonal trees," *IEEE Trans. Comput.,* vol. C-32, pp. 569–581, June 1983.

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.

[19] C. H. Sequin, "Doubly twisted torus networks for VLSI processor arrays," in *Proc. 8th Annu. Int. Symp. Comput. Architecture,* 1981, pp. 471–480.
[20] N. G. de Bruijn, "A combinatorial problem," Koninklijke Netherlands: Academe Van Wetenschappen, *Proc. Vol. 49,* part 20, 1946, pp. 758–764.
[21] M. L. Schlumberger, "de Bruijn communications networks," Ph.D. dissertation, Dep. Comput. Sci. Stanford Univ., 1974.
[22] M. Imase and M. Itoh, "Design to minimize diameter on building-block networks," *IEEE Trans. Comput.,* vol. C-30, pp. 439–443, June 1981.
[23] D. K. Pradhan and S. M. Reddy, "A fault-tolerant communication architecture for distributed systems," *IEEE Trans. Comput.,* vol. C-31, pp. 863–870, Sept. 1982.
[24] M. R. Garey and D. S. Johnson, *Computers and Intractability—A Guide to the Theory of NP-Completeness.* San Francisco, CA: Freeman, 1979.
[25] T. Etzion and A. Lempel, "Algorithms for the generation of full-length shift-register sequences," *IEEE Trans. Inform. Theory,* vol. IT-30, pp. 244–251, May 1984.
[26] C. D. Thompson, "A complexity theory for VLSI," Ph.D. dissertation, Carnegie-Mellon Univ., Dep. Comp. Sci., Aug. 1980.
[27] L. Synder, "Introduction to the configurable highly parallel computer," *IEEE Computer,* pp. 47–56, Jan. 1982.
[28] J. L. Bentley, "Decomposable searching problem," *Inform. Processing Lett.,* vol. 8, pp. 244–251, June 1979.
[29] H. T. Kung, "Let's design algorithms for VLSI systems," in *Proc. Conf. Very Large Scale Integration: Architecture, Design and Fabrication, at California Institute of Technology,* Jan. 1979.
[30] L. Haynes, R. Lau, D. Siewiorek, and D. Mizell, "A survey of highly parallel computing," *IEEE Computer,* pp. 9–24, Jan. 1982.
[31] F. T. Leighton and C. E. Leiserson, "Wafer-scale integration of systolic arrays," in *Proc. 23rd IEEE Symp. Foundations Comput. Sci.,* pp. 297–311.
[32] P. Banerjee and J. A. Abraham, "Fault-secure algorithms for multiple-processor system," in *Proc. 11th Int. Symp. Comput. Architecture,* June 1986, pp. 279–287.
[33] C. L. Seitz, "The cosmic cube," *Commun. ACM,* vol. 28, pp. 22–33, Jan. 1985.
[34] A. Esfahanaian and S. L. Hakimi, "Fault-tolerant routing in the de Bruijn communication networks," *IEEE Trans. Comput.,* vol. C-34, pp. 777–788, Sept. 1985.
[35] F. T. Leighton, "Optimal layouts for the shuffle-exchange graph and lower bound techniques for VLSI," Ph.D. dissertation, M.I.T., Aug. 1981.
[36] J. I. Raffel *et al.,* "a wafer-scale integrator," in *Proc. Int. Conf. Comput. Design,* Oct. 1984, pp. 212–126.
[37] A. K. Somani and V. K. Agarwal, "An efficient VLSI dictionary machine," in *Proc. Int. Conf. Comput. Architecture,* June 1984, pp. 142–150.
[38] M. R. Samatham and D. K. Pradhan, "A multiprocessor network suitable for single-chip VLSI implementation," in *Proc. 11th Symp. Comput. Architecture,* June 1984, pp. 328–337.
[39] M. Ajtai, J. Komlos, and E. Szemeredi, "An $O(N \log N)$ sorting networks," in *Proc. 15th ACM Symp. Theory Comput.,* 1983, pp. 1–9.
[40] K. E. Batcher, "Sorting networks and their applications," in *Proc. 1968 SJCC,* AFIPS, vol. 32, 1968, pp. 307–314.
[41] T. C. Chen, V. Y. Lum, and C. Tung, "The rebound sorter: An efficient sort engine for large files," in *Proc. 1978 Int. Conf. Very Large Data Bases,* 1978, pp. 312–318.
[42] D. E. Knuth, "Sorting and searching," *The Art of Computer Programming, Vol. 3.* Reading, MA: Addison-Wesley, 1973.
[43] F. T. Leighton, "Tight bounds on the complexity of parallel sorting," in *Proc. 16th ACM Symp. Theory Comput.,* 1984, pp. 71–80.
[44] D. E. Muller and F. P. Preparata, "Bounds to complexities of networks for sorting and switching," *J. ACM,* vol. 22, pp. 196–201, 1975.
[45] D. Nassimi and S. Sahni, "Parallel permutation and sorting algorithms and a new generalized connection network," *J. ACM,* vol. 29, pp. 642–667, July 1982.
[46] C. D. Thompson, "The VLSI complexity of sorting," *IEEE Trans. Comput.,* vol. C-32, Dec. 1983.
[47] L. E. Winslow and Y.-C. Chow, "The analysis and design of some new sorting machines," *IEEE Trans. Comput.,* vol. C-32, pp. 677–683, July 1983.
[48] H. Yasuura, N. Takagi, and S. Yajima, "The parallel enumeration sorting scheme for VLSI," *IEEE Trans. Comput.,* vol. C-31, pp. 1192–1201, Dec. 1982.
[49] F. T. Leighton, private communication.
[50] M. R. Samatham and D. K. Pradhan, "De Bruijn multiprocessor network: A versatile sorting network," in *Proc. 12 Int. Symp. Comput. Architecture,* 1985, pp. 360–367.



Maheswara R. Samatham (S'82–M'85) was born in Vijayawada, India in December 1956. He received the B.Tech. degree in electronics and communication engineering from Jawaharlal Nehur Technological University, the M.Tech. degree in electrical engineering from the Indian Institute of Technology, Kanpur, and the Ph.D. degree in electrical and computer engineering from the University of Massachusetts, Amherst.

Since January 1986, he has been with Northwestern University, Evanston, IL, as an Assistant Professor in the Department of Electrical Engineering and Computer Science. His current research interests include computer architecture, parallel processing, VLSI/WSI computation, and fault-tolerant computing.

Dr. Samatham is a member of the IEEE Computer Society, the Association for Computing Machinery, and Tau Beta Pi.

Dhiraj K. Pradhan (S'70–M'72–SM'80–F'88), for a photograph and biography, see p. 365 of the March 1989 issue of this TRANSACTIONS.

Authorized licensed use limited to: Lakehead University. Downloaded on September 24,2026 at 15:23:45 UTC from IEEE Xplore. Restrictions apply.